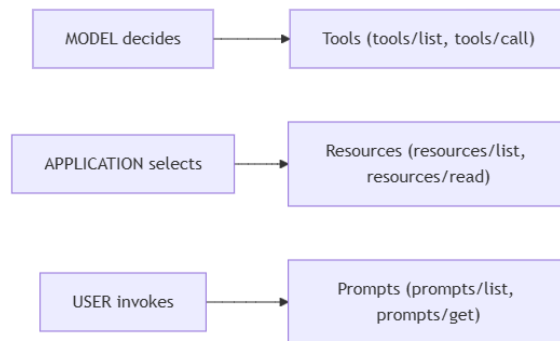


Server Primitives Deep Dive Objectives

- Implement all three server primitives — **Tools / Resources / Prompts**
- Choose **direct vs templated resources**;
- Expose **prompts as slash commands**;
- Validate I/O with **Pydantic**;
- Decide between **FastMCP** and the **Low-Level `Server` API**.

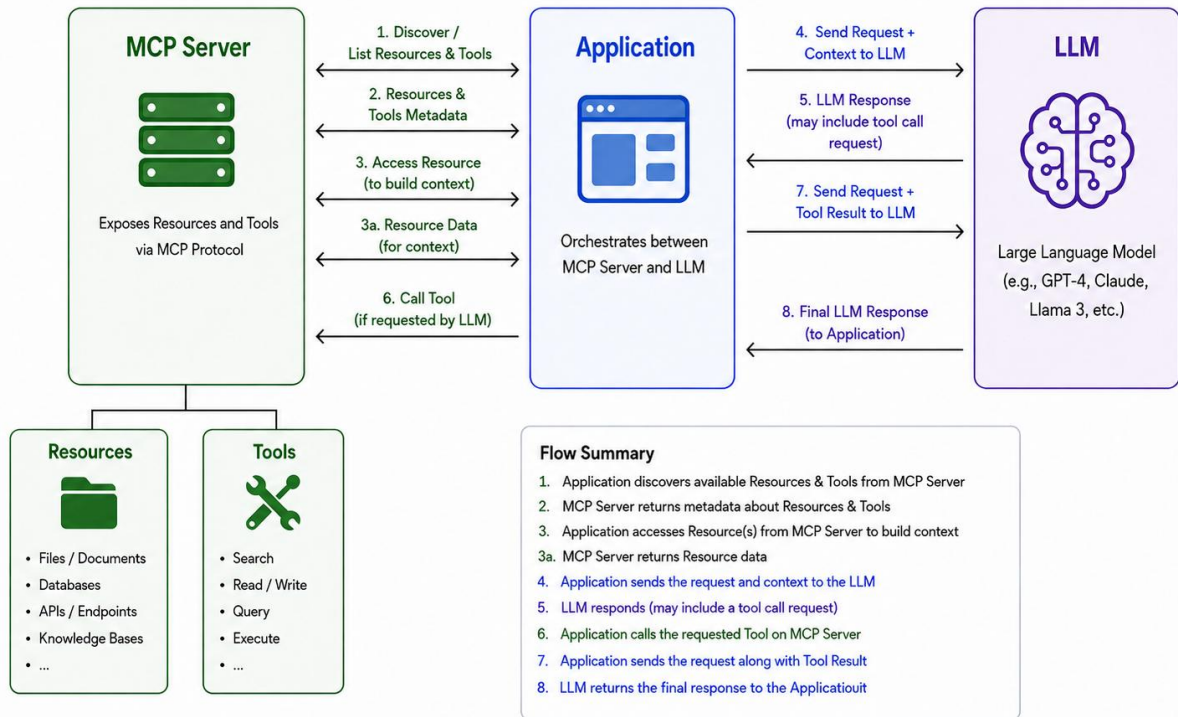
About Three Primitives

1. The Three Primitives (who controls what)



App to LLM = Prompt + Context (Call to resources of MCP Servers) + Tools (Name+Description)

MCP Server, Application and LLM Working Together



What are Tools

Tools are **actions** — functions the AI can call to *do* something. They are dynamic, may have side effects, and return results based on the inputs provided.

Characteristics:

- Invoked by the model with specific arguments
- Can read, write, create, delete, or trigger external operations
- Results are ephemeral (not persistent in the protocol)

Methods: `tools/list`, `tools/call`

Use Cases:

- **Search the web** — `search(query: "latest AI news")` → returns search results
- **Send an email** — `send_email(to: "...", subject: "...", body: "...")` → delivers email
- **Query a database** — `run_sql(query: "SELECT * FROM orders WHERE status='pending'")` → returns rows
- **Create a file** — `write_file(path: "report.docx", content: "...")` → saves file to disk
- **Call an API** — `get_weather(city: "Hyderabad")` → returns current weather

Think of a Tool like a **function call** — you pass inputs, something happens, you get output.

What are Resources?

Resources are **data** — named, addressable content that the AI can *read*.

They are passive, read-only, and identified by a URI.

Methods: resources/list, resources/templates/list, resources/read, resources/subscribe (monitor changes). Application-driven (tree views, search, auto-context).

Two discovery patterns: Direct and templated:

a) **Direct Resources** — each has a unique **URI** and a **MIME type**.

- **Direct resource** — fixed URI (`calendar://events/2024`).

Characteristics:

- Identified by a URI (e.g., `file:///docs/policy.pdf`, `db://customers/1234`)
- Read-only — the model fetches them, doesn't mutate them
- Can be static (a file) or dynamic (a database record fetched at read time)

Example:

```
@mcp.resource("file:///docs/policy.pdf")
def get_policy_doc() -> str:
    """Returns the content of policy.pdf"""
    ...
```

b) **Resource Template:**

Parameterized URI (`travel://activities/{city}/{category}`), supporting **parameter completion** (typing "Par" → "Paris").

Example:

```
@mcp.resource("greeting://{name}")
def get_greeting(name: str) -> str:
    """Get a personalized greeting"""
    return f"Hello, {name}!"
```

Use Cases of Resources:

- **Company knowledge base** — `file:///docs/hr-policy.md` → Claude reads the HR policy to answer employee questions

- **Customer record** — `db://customers/C001` → Claude reads a customer profile to personalize a response
- **Code files** — `file:///src/app.py` → Claude reads source code for review or debugging
- **Configuration** — `config://app/settings.json` → Claude reads app config before generating a deployment checklist

Think of a Resource like a **document or record** — it has an address, you fetch it, you read it.

Tools vs Resources

Key Differences:

	Tools	Resources
Nature	Action / function	Data / content
Side effects	Yes (can write/trigger)	No (read-only)
Identified by	Name + schema	URI
Example	<code>send_slack_message(...)</code>	<code>slack://channels/general/history</code>

Note: The difference isn't about read vs. write — it's about **addressed content** vs. **invoked computation**.

Eg: Search the web — `search(query: "latest AI news")` → **returns search results**

Question: Why this should be tool and not a resource?

Answer: MCP's design treats anything that takes dynamic input and executes logic as a Tool.

Resources are for **content** you can *point at*; Tools are for **operations** you need to *run*.

Web search breaks rules:

1. **No stable URI** A resource URI like `file:///docs/policy.md` points to one specific thing. But `search("latest AI news")` isn't a fixed location — it's a *computation*. You can't pre-define a URI for every possible search query a user might ask.
2. **The intent is to *invoke* something, not *fetch* something.** Resources are passive — you retrieve data that already exists somewhere. A web search actively goes out, hits a search engine, aggregates results from multiple sources, and returns a freshly computed response. That's inherently a function call, not a data fetch.

Think of it this way:

- `file:///news/ai-weekly.md` → a Resource (a specific document you can fetch)
- `search("latest AI news")` → a Tool (a query you execute against a live engine)

What are Prompts

Prompts are **pre-defined, reusable message templates** that an MCP server exposes to the client. They tell the AI *how to behave* or *what to say* in a specific situation.

Prompt templates are prompts with **dynamic placeholders** — variables that get filled in at invocation time.

For example, a template below becomes a reusable prompt where `document_type`, `num_points`, and `content` are arguments passed when invoking it.

Summarize the following `{{document_type}}` in `{{num_points}}` bullet points: `{{content}}`

- Surfaced as **slash commands** (`/plan-vacation`), command palettes, or buttons.
- Methods: `prompts/list`, `prompts/get`.
- Can reference resources/tools to build whole workflows.

How it works in practice

1. The MCP server **registers** a named prompt with an optional argument schema
2. The client **discovers** available prompts (just like it discovers tools)
3. The client **invokes** the prompt by name + arguments
4. The server **returns** the fully rendered message(s), ready to send to the model

Prompt Use Cases:

Prompts showing the template with variables:

1. Code Review

prompt: "code-review", args: { language: "Python", code: "..." }

Prompt template:

You are a senior `{{language}}` developer. Review the following code strictly for:

1. PEP8 compliance
2. Unused imports
3. Missing error handling
4. Logic errors

Code:

`{{code}}`

Return findings as: [ISSUE TYPE] - [LINE NUMBER] - [DESCRIPTION]

2. Customer Support Reply

prompt: "support-reply", args: { issue_type: "billing", customer_name: "Ravi" }

Prompt template:

You are a polite support agent for Deccansoft.

The customer's name is {{customer_name}} and their issue is about {{issue_type}}.

Rules:

- Always greet by name
- Never promise refunds without escalation approval
- End with a case reference number placeholder

Draft a reply following these rules.

3. SQL Generator

prompt: "generate-sql", args: { table_schema: "...", user_question: "..." }

Prompt template:

Given this table schema:

{{table_schema}}

Write a single, optimized SQL query to answer:

"{{user_question}}"

Rules:

- Use standard SQL only
- No subqueries unless necessary
- Add a comment explaining the logic

Why Prompts as a first-class MCP primitive?

Without MCP Prompts	With MCP Prompts
Prompt logic scattered across clients	Centralized on the server
Each client reimplements the same prompt	One definition, many clients reuse it
Hard to update prompts across tools	Update once on the server, all clients get it
No discoverability	Clients can list available prompts dynamically

What Real Servers Actually Expose

A recurring student misconception is that every server offers all three primitives. In practice **production servers are overwhelmingly tools-heavy**; resources and especially prompts are rarer and have distinct idioms. Real inventories from the official READMEs:

1. Server: filesystem (@modelcontextprotocol/server-filesystem)

- **14 Tools:** read_text_file, read_media_file, write_file, edit_file, create_directory, list_directory, move_file, search_files, directory_tree, get_file_info, list_allowed_directories, ...

2. git (mcp-server-git)

- **12 Tools:** git_status, git_diff_unstaged, git_diff_staged, git_diff, git_commit, git_add, git_reset, git_log, git_create_branch, git_checkout, git_show, git_branch

3. fetch (mcp-server-fetch)

- **1 Tool:** fetch(url, max_length, start_index, raw)
- **1 Prompt:** fetch (arg: url) → "Fetch a URL at {{url}} and extract its contents as markdown"

4. GitHub (github/github-mcp-server)

- **Many Tools:** Tools are grouped into **toolsets**: context, repos, issues, pull_requests, actions, code_security, discussions, users

Hands-On Lab — A Multi-Primitive Server for Institute

Two servers, each teaching something the other can't:

Example:

1. git clone <https://github.com/DeccansoftAITeam/MCPDemos.git>
2. cd **ServerPrimitiveDemo**
3. Follow Instructions in SERVER_GUIDE.md
4. To run client with STDIO: (**Follow instructions in CLIENT_GUIDE.md**)
 - a. `venv\Scripts\activate`
 - b. `python stdio_client.py`
5. To run the client with HTTP:
 - a. **Terminal 1:**

```
python main.py http
# Starting MCP server on http://127.0.0.1:8000/mcp ...
```
 - b. **Terminal 2:**

```
python http_client.py
```

Tool Descriptor In-depth

Most people learn name + description + inputSchema and stop. The 2025-06-18 spec defines six fields, and the last three are where quality lives.

```
{
  "name": "search_products",
  "title": "Search Products",
  "description": "Search the product catalog by keyword...",
  "inputSchema": { "type": "object", "properties": { ... }, "required": [...] },
  "outputSchema": { "type": "object", "properties": { ... } },
  "annotations": { "readOnlyHint": true, "openWorldHint": false }
}
```

Field	Who reads it	Why it matters
name	The model	This is the identifier the model emits in a tool call
title	The human	Shown in approval dialogs and tool pickers. Some clients drop tools with no title
description	The model	The single biggest determinant of whether the tool gets called correctly
inputSchema	Model + server	Model uses it to construct arguments; server uses it to validate
outputSchema	Client + model	Enables validation and typed handling of results
annotations	The client/host	Behavioural hints that drive approval UX

Example:

```
from typing import Annotated, Literal
from pydantic import BaseModel, Field
from fastmcp import FastMCP

mcp = FastMCP("catalog")
```

```

class Product(BaseModel):
    # drives outputSchema
    id: int = Field(description="Stable product identifier")
    name: str = Field(description="Display name")
    price_inr: float = Field(description="Unit price in INR, excluding GST")

@mcp.tool(
    name="search_products",
    title="Search Products",
    annotations={
        "readOnlyHint": True,
        "destructiveHint": False,
        "idempotentHint": True,
        "openWorldHint": False,
    },
)
def search_products(
    #inputSchema
    query: Annotated[str, Field(description="Free-text keywords over name and description.",
min_length=2, max_length=100)],
    category: Literal["books", "electronics", "grocery"] | None = Field(None, description="Restrict to
one category. Omit to search all."),
    limit: int = Field(10, description="Maximum rows returned.", ge=1, le=50),
) -> list[Product]:
    # ← outputSchema
    """Search the product catalogue by free-text keyword.

    Use when the user describes a product in words.

    Do NOT use when you already have a product ID - call get_product instead.
    """
    # ← description
    return []

```

What the client actually receives

```

{
  "name": "search_products",
  "title": "Search Products",

```

```
"description": "Search the product catalogue by free-text keyword.\n\nUse when the user describes a product in words. Do NOT use when you already\nhave a product ID - call get_product instead.",

```

```

"items": {
  "type": "object",
  "properties": {
    "id": { "type": "integer", "description": "Stable product identifier" },
    "name": { "type": "string", "description": "Display name" },
    "price_inr": { "type": "number", "description": "Unit price in INR, excluding GST" }
  },
  "required": ["id", "name", "price_inr"]
}
},
"required": ["result"],
"x-fastmcp-wrap-result": true
},
"annotations": {
  "readOnlyHint": true,
  "destructiveHint": false,
  "idempotentHint": true,
  "openWorldHint": false
}
}

```

About Naming and Titles:

Rules:

- snake_case, verb-first, unambiguous: create_invoice, not invoice or doInvoice
- Prefix by domain when a server exposes many tools: crm_find_contact, crm_create_contact.
This also survives multi-server setups where two servers both expose search
- Avoid near-duplicates. get_user and fetch_user on the same server guarantees wrong picks
- Keep under ~40 chars; some hosts truncate

X Ambiguous — the model cannot tell these apart

@mcp.tool

def data(id: str): ...

@mcp.tool

def get_data(id: str): ...

```
# ✓ Distinct intent encoded in the name
```

```
@mcp.tool
```

```
def get_order_by_id(order_id: str): ...
```

```
@mcp.tool
```

```
def list_orders_for_customer(customer_id: str): ...
```

Writing Descriptions the Model Actually Uses Correctly

A reusable five-part anatomy students can apply to every tool:

1. **What it does** — one sentence, active voice
2. **When to use it / when NOT to** — the disambiguator
3. **Argument notes the schema can't express** — units, formats, sourcing
4. **What it returns** — shape and size
5. **Failure modes** — what an error means and what to do next

```
@mcp.tool
```

```
def search_products(query: str, category: str | None = None, limit: int = 10) -> list[dict]:
```

```
    """Search the product catalogue by free-text keyword.
```

```
    Use this when the user describes a product in words ("cheap running shoes").
```

```
    Do NOT use this when you already have a product ID — use `get_product` instead,
    which is faster and returns full detail.
```

```
    Results are ranked by relevance, not price. Prices are in INR, excluding GST.
```

```
    Returns at most `limit` rows; if the user needs more, call again with a
    narrower `query` rather than raising `limit`.
```

```
    Raises an error if the search index is unavailable — in that case tell the
    user the catalogue is temporarily unreachable; do not retry more than once.
```

```
    """
```

```
    ...
```

MCPServer takes the docstring as the description automatically, and also parses per-parameter descriptions out of Google/NumPy-style docstrings.

Input Schema Quality

The model can only supply arguments as well as the schema describes them. Push constraints into the schema instead of validating in the body.

```

from typing import Annotated, Literal
from pydantic import Field

@mcp.tool
def search_products(#query, category, max_price, limit)
    query: Annotated[str, Field(
        description="Free-text keywords, 2-100 chars. Do not include category names here.",
        min_length=2, max_length=100)],
    category: Annotated[Literal["books", "electronics", "grocery"] | None, Field(
        description="Restrict to one category. Omit to search all categories.") = None,
    max_price: Annotated[float | None, Field(
        description="Upper price bound in INR, excluding GST.", gt=0)] = None,
    limit: Annotated[int, Field(
        description="Maximum rows returned.", ge=1, le=50)] = 10,
) -> list[dict]: ...

```

What this buys you:

- Literal becomes a JSON Schema enum — the model stops inventing category names
- ge/le become minimum/maximum — no more limit: 10000
- Sensible defaults mean fewer required args, which means fewer malformed calls
- Validation errors come back automatically and are actionable

Output Schemas and structuredContent:

Best fit: Module 5 (concept). Then apply it in Module 9 (return query rows as structured content) and Module 8 (file listings).

Pre-2025-06-18, every tool result was a blob of text. Now a result may carry both:

```

{
  "content": [
    { "type": "text", "text": "{\"temperature\": 22.5, \"conditions\": \"Partly cloudy\"}" }
  ],
  "structuredContent": { "temperature": 22.5, "conditions": "Partly cloudy" }
}

```

Rules from the spec:

- If you declare an outputSchema, your results **MUST** conform to it
- Clients **SHOULD** validate against it
- Output schemas must be "type": "object" at the top level

- For backwards compatibility, also serialise the JSON into a text content block

In MCPServer the return type annotation generates the output schema for free:

```
from pydantic import BaseModel

class Product(BaseModel):
    id: int
    name: str
    price_inr: float
    in_stock: bool

@mcp.tool
def get_product(product_id: int) -> Product:
    """Return full detail for one product by ID."""
    ...
```

Worth calling out explicitly, because it surprises everyone: a **primitive** return type gets wrapped under a result key, since the schema must be an object.

```
@mcp.tool
def calculate_sum(a: int, b: int) -> int: ...
# structuredContent -> {"result": 8}
```

For full control:

```
from fastmcp.tools import ToolResult

@mcp.tool
def sales_report(month: str) -> ToolResult:
    rows = query(month)
    return ToolResult(
        content=f"{len(rows)} orders totalling ₹{total(rows):,.0f} in {month}.", # for the model/human
        structured_content={"month": month, "order_count": len(rows), "total_inr": total(rows)},
        meta={"query_ms": 42},
    )
```

Tool Annotations (Behavioural Hints)

Best fit: "The MCP Indicator & Per-Action Approval" — annotations are what drive that UX.

Four boolean hints tell the host how dangerous a tool is:

Annotation	Meaning	Effect on host
readOnlyHint	Does not modify its environment	May auto-approve, or badge as safe
destructiveHint	May perform destructive updates	Stronger confirmation prompt
idempotentHint	Repeat calls with same args have no extra effect	Safe to retry after a timeout
openWorldHint	Talks to an open world of external entities (the internet)	Signals data-exfiltration surface

```
@mcp.tool(annotations={"readOnlyHint": True, "openWorldHint": False})
def list_tables() -> list[str]: ...

@mcp.tool(annotations={"readOnlyHint": False, "destructiveHint": True,
                      "idempotentHint": False, "openWorldHint": False})
def drop_customer(customer_id: int) -> str: ...

@mcp.tool(annotations={"readOnlyHint": True, "openWorldHint": True})
def fetch_url(url: str) -> str: ...
```

The critical caveat, straight from the spec: clients **MUST** treat annotations as untrusted unless the server is trusted. They are *hints*, not enforcement. A malicious server can declare readOnlyHint: true on a tool that deletes your database.

Error Handling: Two Mechanisms, Two Meanings

Best fit: Module 5 (concept) + Module 9 (your allow-list rejections are the ideal worked example).

MCP has two distinct error channels and choosing wrong breaks the agent loop.

Protocol errors — JSON-RPC error object. Means *the request itself was invalid*. The model generally cannot recover.

```
{ "jsonrpc": "2.0", "id": 3,
  "error": { "code": -32602, "message": "Unknown tool: invalid_tool_name" } }
```

Tool execution errors — a normal result with isError: true. Means *the tool ran and failed*. The model sees this and can adapt.

json

```
{ "jsonrpc": "2.0", "id": 4,
  "result": { "content": [{
```

```

        "type": "text",
        "text": "Failed to fetch weather data: API rate limit exceeded. Retry
after 60s."
    },
    "isError": true
}

```

Rule of thumb:

- Unknown tool, malformed arguments, server crash → protocol error
- Row not found, upstream API 500, business rule violated, permission denied → isError: true with a message the model can act on

In FastMCP, exceptions become error results automatically. Use ToolError when you want a specific message to reach the client, and mask_error_details=True on the server when you don't want internal exception text leaking:

```

from fastmcp.exceptions import ToolError
mcp = FastMCP("orders", mask_error_details=True)
@mcp.tool
def cancel_order(order_id: int) -> str:
    order = db.get(order_id)
    if order is None:
        # Reaches the client verbatim, even with masking on
        raise ToolError(f"No order with id {order_id}. Use list_orders to find valid IDs.")
    if order.status == "shipped":
        raise ToolError("Order already shipped and cannot be cancelled. "
                        "Offer the customer a return instead.")
    ...

```

Note: error messages are prompts too.

Compare:

BAD - "Error: 403"

GOOD - "Permission denied: this API token is read-only. Do not retry; tell the user they need write access provisioned."

Note: The second one stops the agent from burning five turns retrying.

Tool Granularity: How Many Tools, and How Big

Three modes to name and contrast:

1. **BAD - The God Tool** — one tool, a mode argument, twelve branches. The schema can't describe it, so the model guesses.

```
# Wrong Approach
@mcp.tool
def crm(action: str, payload: dict) -> dict: ...
```

2. **BAD - The CRUD Sprawl** — a 1:1 mirror of your REST API. 60 tools, each doing one HTTP call, so the model needs six calls to answer one question.
3. **GOOD - The Workflow Tool** — sized to a *user intent*, not an endpoint. This is the target.

```
# One call answers a real question
@mcp.tool
def get_customer_summary(customer_id: int) -> CustomerSummary:
    """Return a customer's profile, last 5 orders, and open support tickets."""
```

Guidance for using tools:

- Size tools to intents, not endpoints. If the model always calls A then B, ship one tool
- Aim for a dozen or so well-named tools per server; past ~40 selection accuracy visibly degrades
- Every tool definition costs tokens on every turn — a 60-tool server can spend thousands of tokens before the user has typed anything
- Tools that only ever return data the model reads should probably be a **Resource** instead.

Result-size discipline

```
# Wrong Approach - Returns 4 MB of JSON, blows the context window
@mcp.tool
def get_all_orders() -> list[dict]: ...

# ✓ Paginated, capped, with a pointer to the bulk data
@mcp.tool
def list_orders(customer_id: int, limit: int = 20, cursor: str | None = None) -> OrderPage:
    """Return up to `limit` orders. Use `next_cursor` to page. Never returns more than 50."""
```

For genuinely large payloads, return a `resource_link` instead of the bytes:

```
{
  "type": "resource_link",
  "uri": "file:///reports/sales-2026-08.csv",
  "name": "sales-2026-08.csv",
```

```
"mimeType": "text/csv",
"description": "Full 40k-row export"
}
```

The model gets a handle; the client fetches it only if needed.

Changing Tools Safely

Clients cache the tool list after tools/list.

If your tool set changes at runtime, announce it:

```
{ "jsonrpc": "2.0", "method": "notifications/tools/list_changed" }
```

This requires declaring the capability at initialize time:

```
{ "capabilities": { "tools": { "listChanged": true } } }
```

Compatibility rules worth stating plainly:

- Adding an optional parameter → safe
- Adding a required parameter → breaking
- Renaming a tool → breaking. Keep the old name as an alias whose description says "Deprecated, use X"
- Tightening an enum or a range → breaking
- Loosening a constraint → safe

The capability is automatic: You don't hand-write {"capabilities": {"tools": {"listChanged": true}}}

Any FastMCP server that registers tools declares it during initialize

Sending the notification

In FastMCP 3.x, mutating the tool set does **not** by itself emit **notifications/tools/list_changed**. You send it explicitly through the session on the **Context**.

```
import asyncio

from mcp.server.mcpserver import MCPServer, Context


mcp = MCPServer("dyn")


@mcp.tool()
def dummy_tool() -> str:
    """A simple tool that returns a string."""
```

```

    return "Hello from the dummy tool!"

def beta_feature() -> str:
    """A tool that only exists once the user opts in."""
    return "beta"

@mcp.tool()
async def enable_beta(ctx: Context) -> str:
    """Turn on the beta toolset for this session."""

    # MCPServer.add_tool() accepts the function and builds the Tool wrapper.
    mcp.add_tool(beta_feature)

    await ctx.session.send_tool_list_changed()  # ← the announcement

    return "Beta tools enabled."

if __name__ == "__main__":
    mcp.run(transport="stdio")

```

Other methods on ctx.session: `send_resource_list_changed()`, `send_prompt_list_changed()`.